

Systèmes d'exploitation I

Chapitre 4 : Communication interprocessus

Fichiers et tubes

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

Sommaire

- 1 Introduction
- 2 Interface de programmation
- 3 Flots d'entrée/sortie et tubes
- 4 Conclusion

Objectifs :

- Comprendre la logique des fichiers et des descripteurs.
- Manipuler ces fichiers avec l'API POSIX (open, read, write, etc.).
- Comprendre le fonctionnement des tubes (pipes), leur usage en C et en shell.

Introduction

Fichiers et SGF

Un fichier est un ensemble d'informations regroupées sous un nom, conservés par l'OS pour un usage ultérieur → **Système de gestion de fichiers (SGF)**

Fonctions d'un SGF

- **Conservation permanente** des fichiers sur un support de stockage permanent, **ex.** disque
- **Organisation logique** : arborescence, noms, chemins, ...
- **Partage et protection** : droits d'accès, permissions, ...
- **Réalisation des fonctions d'accès** aux fichiers via des opérations standardisées : ouvrir, lire, écrire, fermer, ...

Introduction

Dans Unix "tout est fichier"

- Fichiers classiques : données et programmes exécutables
- Communication avec l'utilisateur : fichiers de config, `stdin`, `stdout`
- Communication entre processus : sockets, tubes, etc.
- Interface du noyau : `/proc`
- Interface avec le matériel : périphériques dans `/dev`

→ Convention: le suffixe indique la nature du contenu

ex. : `.c` (programme C), `.o` (binaire translatable, cf. plus loin), `.h` (entête C), `.gif` (un format d'images), `.pdf` (Portable Document Format), etc.

→ Remarque: ne pas faire une confiance aveugle à l'extension (`man file`)

Interfaces des fichiers

Interface Shell - 1A COLD

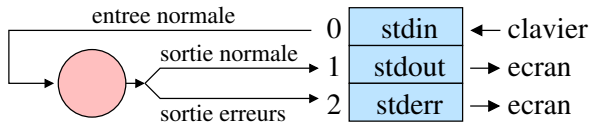
Interface Librairie C - 1A C

→ Ici : Interface système (syscalls)

Interface de programmation POSIX des fichiers

Notion de descripteur - file decriptor

- **Définition** : un petit entier, fourni par le noyau, qui identifie un fichier ouvert.
- **Table des descripteurs** : chaque processus a sa propre table avec 3 descripteurs (**flots d'entrée/sortie**) standards créés à la naissance du processus :
 - ▶ **stdin**: **entrée standard** (clavier)
 - ▶ **stdout**: **sortie standard** (écran)
 - ▶ **stderr**: **sortie d'erreur** (écran)
- Ces flots sont associés aux descripteurs 0, 1 et 2



Interface de programmation POSIX des fichiers

Ouvrir et fermer des fichiers

- `fd = open(path, flags, mode)` → un nouveau descripteur (`fd`) est obtenu ou -1 en cas d'erreur (`errno`)
- `close(fd)` libère un descripteur (`fd`), le rend réutilisable.
- `flags` (obligatoire - comment j'ouvre)
 - ▶ le mode d'ouverture (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) avec options éventuelles
 - ex. `O_CREAT` pour créer le fichier s'il n'existe pas.
- `mode` (optionnel- avec quelles permissions je crée)
 - ▶ spécifie les permissions (`chmod`) à appliquer uniquement si on crée le fichier seulement ie `O_CREAT` est présent dans les flags.
 - ex. `fd = open("/home/toto/titi", O_RDONLY, 0);`

→ `man 2 open` pour plus de détails

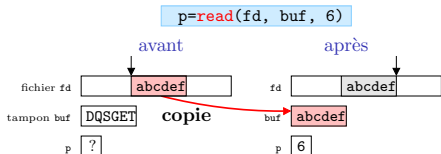
Interface de programmation POSIX

Lire - read()

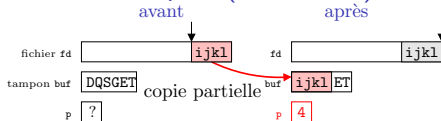
L'objet fichier

- Chaque fichier a un **pointeur courant**: tête de lecture/écriture logique
- Les lectures et écritures le déplacent implicitement
- On peut le déplacer explicitement (avec **lseek()** – voir plus loin)

Lecture normale (read())



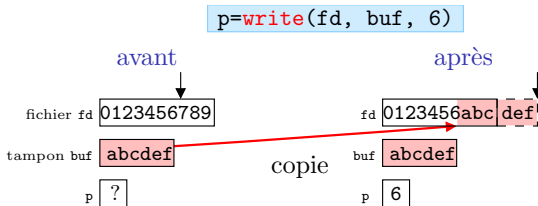
S'il n'y a pas assez de données : lecture tronquée (*short read*)



Interface de programmation POSIX

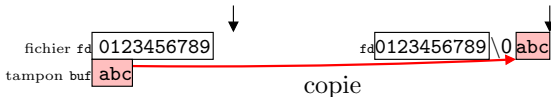
Écrire - write()

- Écriture dans les conditions normales:



Le fichier a été rallongé

- Si un `lseek()` a déplacé le pointeur après la fin, remplissage avec des zéros (qui ne sont pas immédiatement alloués par le SGF)



- Possibilité d'écriture tronquée (*short-write*) si le disque est plein par exemple

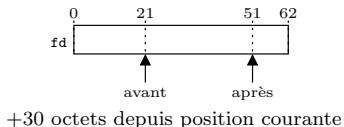
Interface de programmation POSIX

Déplacer la tête de lecture/écriture - `lseek()`

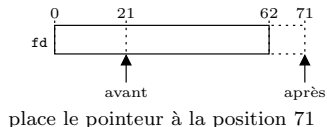
Objectif : déplacer le pointeur de fichier explicitement

Exemples

`lseek(fd, 30, SEEK_CUR)`



`lseek(fd, 71, SEEK_SET)`



- Le pointeur peut être placé après la fin du fichier

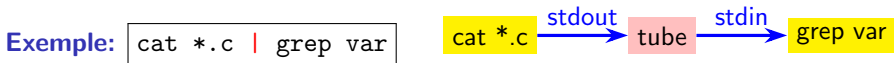
Flots et interface de commande

→ Rediriger les flots en langage de commande: < et >

```
cat fich # écrit le contenu de fich sur la sortie standard (écran)
cat fich > fich1 # copie fich dans fich1 (créé s'il n'existe pas)
./prog < in > out # exécute ./prog
                # en lisant ses données depuis le fichier in (au lieu du clavier)
                # et en écrivant ses résultats dans le fichier out (au lieu de l'écran)
```

→ Tubes (*pipes*): moyen de communication inter-processus

Un tube branche la sortie standard d'un processus sur l'entrée standard d'un autre



- crée deux processus : `cat *.c` et `grep var`
- connecte la sortie du premier à l'entrée du second à travers un tube

→ Que font les commandes suivantes ?

```
cat f1 f2 f3 | grep toto | wc -l > resultat
```

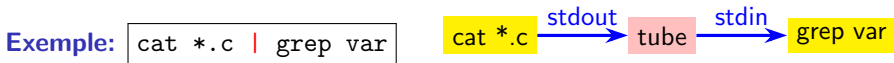
Flots et interface de commande

→ Rediriger les flots en langage de commande: `<` et `>`

```
cat fich # écrit le contenu de fich sur la sortie standard (écran)
cat fich > fich1 # copie fich dans fich1 (créé s'il n'existe pas)
./prog < in > out # exécute ./prog
                  # en lisant ses données depuis le fichier in (au lieu du clavier)
                  # et en écrivant ses résultats dans le fichier out (au lieu de l'écran)
```

→ Tubes (*pipes*): moyen de communication inter-processus

Un tube branche la sortie standard d'un processus sur l'entrée standard d'un autre



- crée deux processus : `cat *.c` et `grep var`
- connecte la sortie du premier à l'entrée du second à travers un tube

→ Que font les commandes suivantes ?

```
cat f1 f2 f3 | grep toto | wc -l > resultat
```

* compte les occurrences de toto dans les trois fichiers, et écrit le décompte dans le fichier resultat

```
./prog 1> f1.txt 2> f2.txt
```

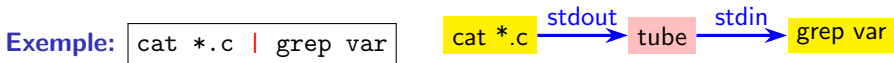
Flots et interface de commande

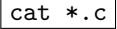
→ Rediriger les flots en langage de commande: < et >

```
cat fich # écrit le contenu de fich sur la sortie standard (écran)
cat fich > fich1 # copie fich dans fich1 (créé s'il n'existe pas)
./prog < in > out # exécute ./prog
                  # en lisant ses données depuis le fichier in (au lieu du clavier)
                  # et en écrivant ses résultats dans le fichier out (au lieu de l'écran)
```

→ Tubes (*pipes*): moyen de communication inter-processus

Un tube branche la sortie standard d'un processus sur l'entrée standard d'un autre



- crée deux processus :  et 
- connecte la sortie du premier à l'entrée du second à travers un tube

→ Que font les commandes suivantes ?

```
cat f1 f2 f3 | grep toto | wc -l > resultat
```

* compte les occurrences de toto dans les trois fichiers, et écrit le décompte dans le fichier resultat

```
./prog 1> f1.txt 2> f2.txt
```

* redirige séparément sortie normale et erreurs dans f1.txt et f2.txt resp.

Interface de programmation des tubes

Appel système `pipe()`

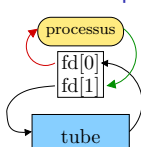
- Crée un tube : deux descripteurs (l'entrée et la sortie du tube)

```
int fd[2];  
pipe(fd);
```

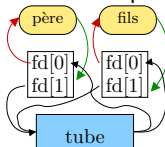
- Ce qui est écrit sur l'entrée (`fd[1]`) est disponible sur la sortie (`fd[0]`) par défaut, cela permet de se parler à soi-même

Mnémotechnique : On lit sur 0 (`stdin`), et on écrit sur 1 (`stdout`)

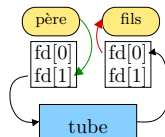
- Application classique : communication père-fils



Après `pipe(fd)`



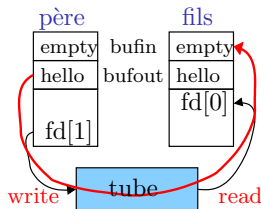
Après `fork()`
les descripteurs sont copiés



Après fermeture des
descripteurs inutiles

Programmation d'un tube père/fils

```
#define BUFSIZE 10
int main(void) {
    char bufin[BUFSIZE] = "empty";
    char bufout[ ] = "hello";
    pid_t childpid;
    int fd[2];
    pipe(fd);
    if (fork() > 0) { /* père */
        close(fd[0]);
        write(fd[1], bufout, strlen(bufout)+1);
    } else { /* fils */
        close(fd[1]);
        read(fd[0], bufin, BUFSIZE);
    }
    printf("[%d]: bufin = %.s, bufout = %s\n",
           getpid(), BUFSIZE, bufin, bufout);
    return 0;
}
```

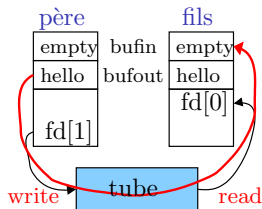


```
$ ./parentwritepipe
[29196]: bufin=empty, bufout=hello
[29197]: bufin=hello, bufout=hello
$
```

Exercice : modifier le programme pour tenir compte des lectures/écritures tronquées

Programmation d'un tube père/fils

```
#define BUFSIZE 10
int main(void) {
    char bufin[BUFSIZE] = "empty";
    char bufout[ ] = "hello";
    pid_t childpid;
    int fd[2];
    pipe(fd);
    if (fork() > 0) { /* père */
        close(fd[0]);
        write(fd[1], bufout, strlen(bufout)+1);
    } else { /* fils */
        close(fd[1]);
        read(fd[0], bufin, BUFSIZE);
    }
    printf("[%d]: bufin = %.*s, bufout = %s\n",
           getpid(), BUFSIZE, bufin, bufout);
    return 0;
}
```



```
$ ./parentwritepipe
[29196]: bufin=empty, bufout=hello
[29197]: bufin=hello, bufout=hello
$
```

Exercice : modifier le programme pour tenir compte des lectures/écritures tronquées

```
int todo = strlen(bufout)+1, done;
char *p=bufout;
while (todo) {
    done = write(fd[1],p,todo);
    todo -= done; p += done; }
```

```
int done=0; bufin[0]='a';
while (bufin[done]) {
    done += read(fd[0],
                bufin+done,
                BUFSIZE-done); }
```


Capacité d'un tube

`write(fd, &BUFF, TAILLE)`: écriture d'au plus **TAILLE** caractères

- S'il n'y a plus de lecteur:
 - ▶ Écriture inutile: on ne peut pas ajouter de lecteurs après la création du tube
 - ▶ Signal SIGPIPE envoyé à l'écrivain (mortel par défaut)
- Sinon si le tube n'est pas plein: Écriture atomique
- Sinon: Écrivain bloqué tant que tous les caractères n'ont pas été écrits (tant qu'un lecteur n'a pas consommé certains caractères)

`read(fd, &BUFF, TAILLE)`: lecture d'au plus **TAILLE** caractères

- Si le tube contient n caractères: Lecture de $\min(n, TAILLE)$ caractères.
- Sinon s'il n'y a plus d'écrivains: Fin de fichier; `read` renvoie 0.
- Sinon: Lecteur bloqué jusqu'à ce que le tube ne soit plus vide (jusqu'à ce qu'un écrivain ait produit suffisamment de caractères)

Opérations non-bloquantes

- Drapeau `O_NONBLOCK` $\Rightarrow$ `read/write` ne bloquent plus jamais:
- Retournent -1 (et `errno=EAGAIN`) si elles auraient dû bloquer
- `fcntl(fd, F_SETFL, fcntl(fd[1], F_GETFL) | O_NONBLOCK);`

Tubes nommés (FIFOs)

- **Problème des tubes**: seulement entre descendants car passage par clonage
- **Solution: tubes nommés** (même principe, mais nom de fichier symbolique)

```
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(char *nom, mode_t mode);
```

- ▶ renvoie 0 si OK, -1 si erreur
 - ▶ mode est numérique (on y revient)
- Après création, un processus l'ouvre en lecture, l'autre en écriture
- Il faut connecter les deux extrémités avant usage (bloquant sinon)
- la *commande* `mkfifo(1)` existe aussi

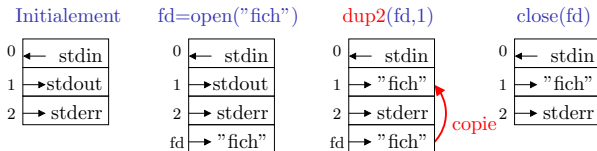
Copie de descripteurs: `dup()` et `dup2()`

```
ls > fich
```

► Cette commande inscrit dans `fich` ce que `ls` aurait écrit.

► Comment ça marche ?

On utilise les appels systèmes `dup()` et `dup2()` pour copier un descripteur :

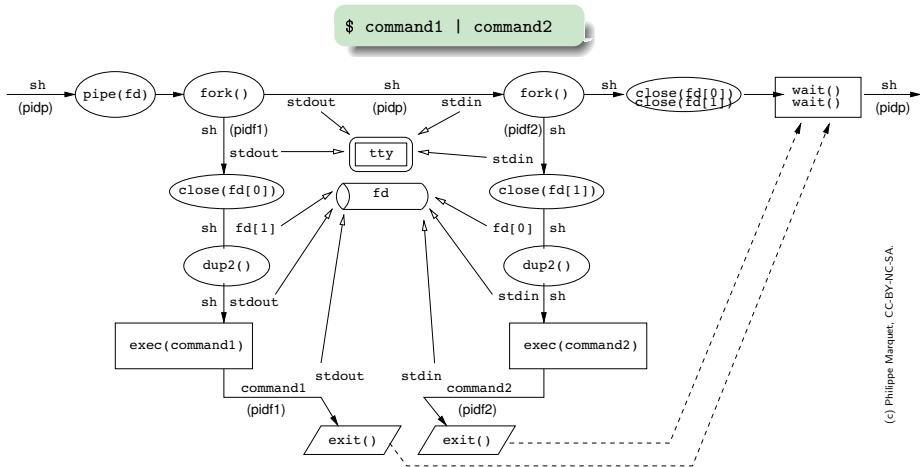


► Appel système `dup(fd)` duplique `fd` dans le premier descripteur disponible

► Appel système `dup2(fd1, fd2)` recopie le descripteur `fd1` dans `fd2`
on dit "dup to", duplicate to somewhere

L'exemple du shell

Création d'un tube entre deux commandes



➔ Si on oublie des `close()`, les lecteurs ne s'arrêtent pas car pour eux il reste des écrivains

Résumé du chapitre

- Tout est fichier sous Unix
- Fichiers et descripteurs comme base de l'I/O Unix.
- API POSIX de bas niveau (syscalls): `open`, `read`, `write`, `close`
Problèmes: `I/O tronquée`, `perfs` par manque de tampon
- Interface standard (bibliothèque) : `fopen`, `fprintf`, `fscanf`, `fclose`
- Trois descripteurs par défaut: `stdin` (0), `stdout` (1), `stderr` (2)
- Rediriger les flots en shell: `<`, `>` et `|`
- Tubes, tubes nommés et redirection en C: `pipe()`, `mkfifo()`, `dup()` et `dup2()`

✓ **Question** : Pourquoi est-ce que les pipes sont considérés comme une des inventions majeures d'Unix ?

Résumé du chapitre

- Tout est fichier sous Unix
- Fichiers et descripteurs comme base de l'I/O Unix.
- API POSIX de bas niveau (syscalls): `open`, `read`, `write`, `close`
Problèmes: I/O tronquée, perfs par manque de tampon
- Interface standard (bibliothèque) : `fopen`, `fprintf`, `fscanf`, `fclose`
- Trois descripteurs par défaut: `stdin` (0), `stdout` (1), `stderr` (2)
- Rediriger les flots en shell: `<`, `>` et `|`
- Tubes, tubes nommés et redirection en C: `pipe()`, `mkfifo()`, `dup()` et `dup2()`

✓ **Question** : Pourquoi est-ce que les pipes sont considérés comme une des inventions majeures d'Unix ?

Ils permettent de chaîner facilement des programmes simples pour réaliser des traitements complexes

→ Philosophie Unix.